

Agentic Architecture Research: Evaluating Single vs. Multi-Agent LLMs Across Static and Interactive Domains

Barta Levente, Mosonyi Martin

June 29, 2026

Abstract

Large Language Models (LLMs) are increasingly deployed as autonomous agents capable of planning, tool use, and multi-step reasoning. While scaling model capability improves performance, the architectural design of the agent system often plays an equally critical role. This report investigates the conditions under which multi-agent LLM architectures outperform single-agent baselines in terms of accuracy and cost-efficiency. We systematically evaluate four architectures—a Single Agent, a Planner-Executor system, an Actor-Critic loop, and a Reflective Memory agent—across two distinct domains: HotpotQA for static, multi-hop reasoning, and MiniWoB++ for dynamic, interactive UI execution. Our methodology isolates the impact of architecture across varying task difficulties, tracking performance metrics alongside token and API call costs to determine the optimal deployment strategies for different task modalities.

1 Introduction

Large Language Models (LLMs) are increasingly used not merely as standalone predictors, but as the central reasoning engines of autonomous agents capable of planning, interacting with external tools, and collaborating within multi-agent networks. As these compound AI systems become more dominant, recent research demonstrates that agent architecture design often has a larger impact on downstream performance and cost-efficiency than underlying model size alone.

The core research question of this study is: *Under what conditions do multi-agent LLM architectures outperform single-agent baselines in terms of accuracy, and cost-efficiency across reasoning and interactive tasks?*

To answer this, we evaluate different agentic configurations across varying difficulty levels in two distinct domains. The first domain, HotpotQA, represents a static, multi-hop reasoning challenge over a fixed set of documents. The second domain, MiniWoB++, represents a dynamic, interactive UI task execution challenge where the environment state changes in response to agent actions. By keeping the foundational LLM constant, we isolate the performance to the architectural topologies themselves.

2 Related Work

The transition from monolithic LLMs to structured agent architectures is motivated by the need to scale reasoning capabilities while controlling for cost and error propagation.

Planner-Executor Paradigms Our Planner-Executor architecture (B1) draws inspiration from *Task Decomposition* frameworks, which argue that LLMs perform better when high-level strategic planning is decoupled from low-level execution [1]. By offloading execution to a specialized executor, we mitigate the tendency for planners to hallucinate tool usage, a common observation in early multi-step reasoning research [2].

Critique and Verification Loops The Actor-Critic framework (B2) aligns with *Self-Correction* research, which demonstrates that LLMs can improve output quality by critiquing their own responses before finalization [3]. Unlike monolithic models that provide a single-shot response, our Critic-led revision loop enforces a verification constraint against the environment state, reducing the rate of premature task completion in interactive domains.

Reflective Learning and Memory Our Reflective/Memory agent (C1) takes high inspiration from *Reflexion* [4], which proposes that agents should not just act, but learn from failure by generating verbal feedback. By consolidating these episodic failures into a frozen list of generic, entity-free rules, our approach extends this work into a test-time memory augmentation strategy, allowing agents to generalize learned strategies across structurally different UI tasks.

3 Methodology

To ensure scientific validity, every architectural variant is evaluated using the exact same underlying LLM (Qwen3.6-27B accessed via API) and on the same input data splits. The implementation relies heavily on the **LangChain** and **LangGraph** frameworks. LangChain is utilized for LLM API abstraction, structured output parsing, and tool binding, while LangGraph manages the complex, cyclic state flows required by multi-agent architectures (such as recursive planning or critic loops). We measure Performance (Exact Match and F1 for text; Average Reward and Action Counts for UI), Cost (Total tokens used and API calls) across the following variables.

3.1 Task Domains and Available Tools

- **HotpotQA (Multi-hop Reasoning):** The agent converts a natural language question into a structured reasoning process over provided Wikipedia passages. To navigate the context, agents are provided with two specific tools: `list_chunks` (to view available document titles and IDs) and `read_content_chunk` (to extract the specific text of a chosen chunk).
- **MiniWoB++ (Interactive UI Tasks):** The agent is given a natural language instruction and a dynamic UI state (DOM elements) to execute a sequence of actions. It is provided with programmatic tools that map directly to browser actions: `click_element`, `type_text`, `press_key`, and `drag`.

3.2 Agent Architectures

We define four distinct agent architectures, modeled as LangGraph state graphs, to be evaluated across both domains:

1. **A1 - Single Agent (Baseline):** A monolithic agent governed by a single system prompt. It is given access to all domain-specific tools and operates in a standard LangChain tool-use loop. It acts iteratively, observing tool outputs, until it produces a final textual answer (HotpotQA) or declares the UI task complete (MiniWoB++).
2. **B1 - Planner → Executor:** A two node LangGraph architecture separating high-level reasoning from low-level tool execution. The *Planner* is blind to the raw source documents or DOM state and has no tools. It uses LangChain’s structured output to issue one narrow sub-task at a time. The *Executor* receives this sub-task, utilizes the available tools to resolve it, and returns a concise observation report. The Planner uses these reports to sequentially build an understanding and eventually output the final solution.
3. **B2 - Actor → Critic:** An iterative verification loop. The *Actor* functions similarly to the Single Agent, using tools to attempt the task. Once it claims to be finished, the state passes to the *Critic*. The Critic evaluates the Actor’s final claim against the initial constraints, the tool trace, and the current environment state. It outputs a structured verdict (`accept` or `reject`). If rejected, the Critic provides precise, actionable feedback, and the LangGraph router redirects flow back to the Actor for a revision step.
4. **C1 - Reflective/Memory Agent:** An extension of the Actor-Critic model that introduces a global learning phase. During training, a *Reflector* agent analyzes failed episodes (e.g., F1 score < 1.0 or reward < 1.0) alongside their full action traces. It creates strategy rules designed to prevent the failure in the future. A *Consolidator* agent then compresses these raw rules into a top-*N* list. These frozen rules are persistently injected into the system prompts of the test-time agents, allowing them to leverage past experience to navigate complex constraints.

4 Experiments

4.1 Datasets

HotpotQA. We use the HotpotQA dataset, where each question comes with ten Wikipedia paragraphs, two of them relevant and the other eight distractors. HotpotQA already labels every question as **easy**, **medium**, or **hard**, and we evaluate the first 500 questions of each level.

MiniWoB++. The environment suite provides only sparse difficulty annotations, so we select 15 representative tasks per difficulty level, covering simple clicks, form filling, drag-and-drop, navigation, and multi-step composite tasks. Each task is run for 3 episodes with fixed reset seeds, giving 45 episodes per difficulty level. Since the reset seed determines the task variant, every architecture encounters the same set of variants.

4.2 Model and Configuration

Every architecture, in both domains, uses the same model (Qwen/Qwen3.6-27B). The tool definitions, the prompts for the shared roles, and the evaluation harness are common to all architectures. For HotpotQA we additionally include a minimal Baseline configuration: a single LLM call with the passages supplied directly in the context and no tool-use loop.

4.3 Memory Training (C1)

The C1 memory bank is generated prior to evaluation on an independent training split and remains frozen during testing. For HotpotQA, this split consists of 100 questions disjoint from the evaluation set. For MiniWoB++, we utilize the same 45 core tasks but initialize them with different reset seeds. Episodes failing to meet the success threshold trigger a reflection phase, generating candidate strategy rules. The Consolidator agent subsequently compresses these into a generalized, entity-free list of the top $N = 7$ rules. These rules are persistently injected into the test-time agents’ system prompts. The computational overhead of this one-time memory generation is accounted for in our cost analysis (Figure 2).

4.4 Metrics

We report two groups of metrics. For performance, HotpotQA uses token-level F1 and Exact Match (EM) against the reference answer, while MiniWoB++ uses the environment’s success rate, together with the average number of actions. For cost, we report token usage and the number of API calls per sample.

5 Results

5.1 Task Performance

Figure 1 reports accuracy across architectures and difficulty levels in the two domains. In HotpotQA, the simpler configurations perform best: the single-call Baseline and the tool-using Single Agent (A1) achieve the highest F1 at every difficulty level, and every multi-agent architecture scores below them. The Actor-Critic (B2) is the weakest, falling to 0.60 F1 on hard questions. In MiniWoB++, the ordering is reversed on hard tasks: the Single Agent (A1) reaches 0.87 reward, whereas the Actor-Critic (B2) and the Memory agent (C1) reach 0.98 and 1.00, respectively.

5.2 Computational Cost

Figure 2 shows the cost per-sample in both domains. In HotpotQA the single-call Baseline uses about 2.2k tokens and one API call per question. The tool-use loop (A1) roughly triples the number of API calls, and the Planner-Executor (B1) is the most expensive of all, at 7 to 9k tokens and 8 to 10 API calls per question. B2 and C1 fall in between. MiniWoB++ shows the same ordering, with B1 again the most expensive, and cost climbs with difficulty as the agent takes more actions, reaching about 20k tokens on the hardest episodes.

Task performance by agent architecture and difficulty

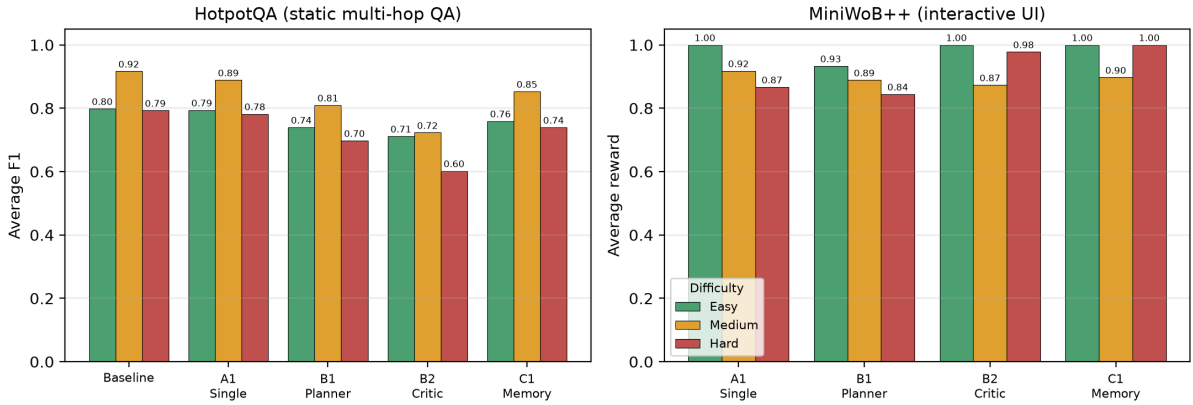


Figure 1: Task performance by architecture and difficulty. **Left:** HotpotQA average F1, where the simpler configurations (Baseline and A1) lead at every level. **Right:** MiniWoB++ average reward, where the critic (B2) and memory (C1) architectures match or beat the single agent on hard tasks.

Table 1: HotpotQA full results (500 questions per difficulty). Tok/s is the average tokens per sample and API/s the average API calls per sample.

Architecture	Difficulty	F1	EM	Tok/s	API/s
Baseline	Easy	0.799	0.673	2197	1.00
Baseline	Medium	0.917	0.846	2160	1.00
Baseline	Hard	0.793	0.635	2262	1.00
A1 Single	Easy	0.794	0.684	2928	3.22
A1 Single	Medium	0.889	0.824	3181	3.52
A1 Single	Hard	0.782	0.642	3193	3.46
B1 Planner	Easy	0.740	0.626	7052	7.75
B1 Planner	Medium	0.810	0.742	8518	9.77
B1 Planner	Hard	0.698	0.566	9299	10.14
B2 Critic	Easy	0.712	0.562	4459	4.13
B2 Critic	Medium	0.723	0.632	4731	4.46
B2 Critic	Hard	0.602	0.444	4686	4.29
C1 Memory	Easy	0.760	0.656	5469	4.05
C1 Memory	Medium	0.853	0.784	5939	4.46
C1 Memory	Hard	0.740	0.586	6117	4.45

6 Analysis

Static reasoning. In HotpotQA the hard part is not the control flow but simply reading the supplied passages and writing down the answer, which the model can already do in one pass. Adding more agents on top does not help with that, and it presents a new problem. The decomposition breaks up the context. The Planner (B1) never sees the raw documents and has to work from short executor reports, so it loses the whole view that a single call has. This matches the data, where B1 is both the most expensive architecture and one of the least accurate. When a task can be solved in one pass, every extra agent adds only cost and risk to hallucinate.

Interactive execution. Conversely, MiniWoB++ requires sequential steps where mistakes easily compound. In this environment, the Actor-Critic (B2) architecture proves highly effective. By acting as a fail-safe against stopping too early, the Critic drives the average reward on hard tasks from 0.87 (A1) up to 0.98 (B2). However, because our Critic only evaluates the final state, it can prevent premature finishes but cannot rescue an episode that has already gone completely off track. Therefore, the performance boost comes primarily from double-checking the final outcome, rather than correcting errors mid-task.

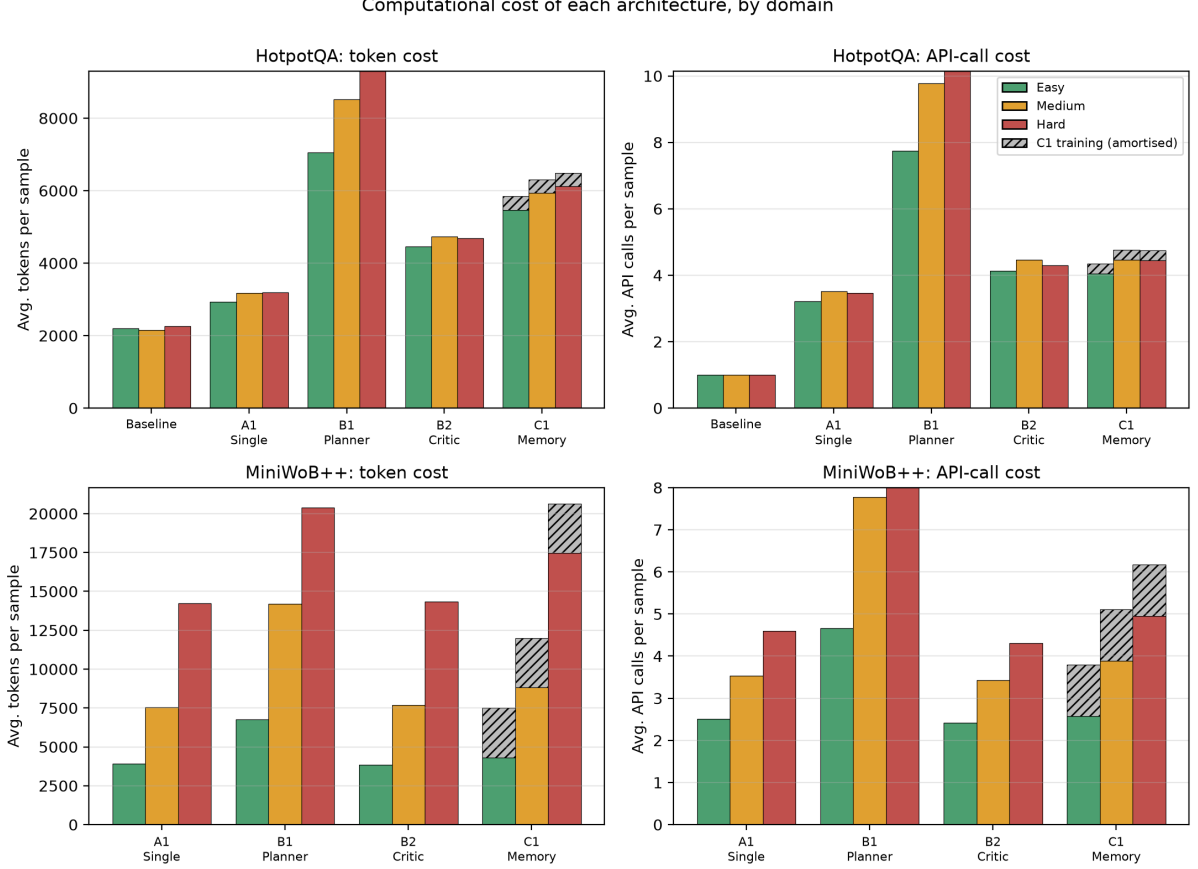


Figure 2: Computational cost per architecture, by domain. **Top:** HotpotQA tokens and API calls per sample. **Bottom:** MiniWoB++ tokens and API calls per sample. The solid bars are inference cost. The hatched cap on the C1 bars is the one-time memory-training cost (Section 4.3) spread evenly over the evaluation set.

Cost. In the static domain, multi-agent configurations utilized two to four times the resources of the Baseline (Figure 2) while yielding inferior F1 scores. In the interactive domain, the cost-benefit analysis is more nuanced. On hard MiniWoB++ tasks, the Critic (B2) achieves a 0.98 reward with negligible overhead compared to A1 (Table 2), as the Critic only expends tokens when an intervention is triggered. While the Memory agent (C1) achieves a perfect 1.00 reward, it incurs higher per-episode costs alongside a substantial one-time training cost (Section 4.3).

Memory. C1 is the one architecture to outperform its Actor-Critic baseline (B2) at test time; however, the absolute gains are marginal. In MiniWoB++, achieving the 1.00 success rate represented solving only a one episode than B2, requiring a memory training phase to generate rules. In HotpotQA, C1 recovers some accuracy lost by the B1 and B2 frameworks (achieving 0.74 against B2’s 0.60 F1 on hard tasks), yet it still fails to surpass the A1 single agent.

Limitations. Several limitations qualify these results.

1. *Metric validity in HotpotQA.* Exact Match, and to a smaller degree F1, care about the exact surface form, but the HotpotQA reference answers are not written consistently. The same kind of answer is accepted in one question and rejected in another. Predicting “Richard Nixon” against a reference of “President Richard Nixon”, for instance, scores $EM = 0$ and $F1 = 0.66$, while in other questions the short form is the one in the reference. This makes the scores look worse than the agents really are, and it leaves the memory with no consistent target.
2. *The static result depends on the benchmark.* The Baseline works only because the ten supplied passages fit inside the model’s context window. If the documents were long enough to overflow the

Table 2: MiniWoB++ full results. Succ is the success rate, Act the average number of actions, Tok/s the average tokens per sample, and API/s the average API calls per sample.

Architecture	Difficulty	Reward	Succ	Act	Tok/s	API/s
A1 Single	Easy	1.000	1.000	1.80	3930	2.51
A1 Single	Medium	0.918	0.956	3.78	7535	3.53
A1 Single	Hard	0.867	0.889	4.40	14241	4.60
B1 Planner	Easy	0.933	0.933	1.87	6756	4.67
B1 Planner	Medium	0.889	0.889	4.13	14181	7.78
B1 Planner	Hard	0.844	0.867	4.24	20389	8.00
B2 Critic	Easy	1.000	1.000	1.80	3860	2.42
B2 Critic	Medium	0.873	0.933	3.80	7670	3.42
B2 Critic	Hard	0.978	0.978	4.27	14324	4.31
C1 Memory	Easy	1.000	1.000	1.80	4324	2.58
C1 Memory	Medium	0.897	0.956	3.89	8809	3.89
C1 Memory	Hard	1.000	1.000	4.31	17454	4.96

context, or if the agent had to retrieve from a large corpus instead of ten shorter passages, a single call would no longer be possible, and some form of navigation or decomposition would be needed. The Baseline’s win should not be read as a general result beyond short, pre-retrieved contexts.

3. *Small interactive sample and small gains.* The MiniWoB++ improvements, while distinct, represent a small absolute number of recovered episodes. Evaluating whether this justifies the compounded inference and training costs requires larger-scale validation.

7 Conclusion

Our research investigated the specific conditions where multi-agent LLMs have an advantage over a standard single agent. The results show a clear split based on the task. In static reasoning tasks where the context is fixed (HotpotQA), adding more agents reduces accuracy and increases token usage. However, in dynamic, multi-step environments (MiniWoB++), features like verification loops (Actor-Critic) and memory systems match or outperform standard single agents. However, because these accuracy gains are small and require significantly more tokens, the multi-agent approach remains a costly trade-off rather than an absolute win.

References

- [1] Wei, J., et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. NeurIPS.
- [2] Yao, S., et al. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR.
- [3] Madaan, A., et al. (2023). *Self-Refine: Iterative Refinement with Self-Feedback*. NeurIPS.
- [4] Shinn, N., et al. (2023). *Reflexion: Language Agents with Verbal Reinforcement Learning*. arXiv preprint.